Ders 10 Genişletilmiş Notu: Eğitmensiz Öğrenme: K-Means, Dirsek Yöntemi ve Hiyerarşik Öbekleme

Ders 10 Genişletilmiş Notu: Eğitmensiz Öğrenme: K-Means, Dirsek Yöntemi ve Hiyerarşik Öbekleme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Eğitmenli ve Eğitmensiz Öğrenme
- 2. Veri Neden Etiketsiz Olur?
- 3. Öbeklemenin Hedefi
- 4. K-Ortalama (K-Means) Algoritması
- 5. Örnek: K-Means'in Elle Çözümü
- 6. Durma Koşulları ve Algoritmanın Akışı
- 7. K-Means'in Sorunları: Başlangıç Merkezleri ve K'nın Seçimi
- 8. Hata Ölçüsü: Küme İçi Kareler Toplamı
- 9. Dirsek Yöntemi ile K'nın Belirlenmesi
- 10. K-Means'in Karmaşıklığı
- 11. Düz ve Hiyerarşik Öbekleme
- 12. Birleştirici Hiyerarşik Öbekleme ve Küme Arası Mesafeler
- 13. Hangi Küme Arası Mesafe Daha İyi?
- 14. Bölücü Hiyerarşik Öbekleme
- 15. Uygulama Alanları
- 16. Önce Öbekle, Sonra Sınıflandır: Veri Analizinin Önemi

1. Eğitmenli ve Eğitmensiz Öğrenme

Hoca, vizeden sonraki bu derste yeni bir öğrenme modeline, eğitmensiz öğrenmeye (unsupervised learning) geçti. Eski uygulamalarda eğitmensiz öğrenmeye çok ihtiyaç duyulmadığını, ama son 4-5 yılda büyük verinin makine öğrenmesiyle çok kullanılmaya başlamasıyla bu modelin en çok öne çıkan öğrenme modellerinden biri olduğunu söyledi.

> **Ders sonrası not:** Hoca girişte "supervised learning'den bahsedeceğim" dedi. Kastettiği unsupervised learning'dir; dersin tamamı eğitmensiz öğrenme üzerinedir.

Önce iki model arasındaki farkı hatırlattı. Bugüne kadar görülen k-en yakın komşu, karar ağacı (decision tree), destek vektör makineleri, perceptron ve çok katmanlı perceptron (multilayer perceptron) eğitmenli öğrenme (supervised learning) yöntemleridir. Hoca, yapay sinir ağlarının eğitmensiz olanlarının da bulunduğunu, bu yüzden ayrım yapmak gerektiğini ekledi.

Eğitmenli öğrenmede elimizdeki veride girişler ve bu girişler için beklenen çıkış değerleri etiketli olarak verilir. Örneğin bir yüz tanıma sisteminde fotoğrafın kime ait olduğu, bir doküman sınıflandırma sisteminde dokümanın magazin haberi mi yoksa spor haberi mi olduğu bilinir. Eğitmensiz öğrenmede ise elimizde yalnızca örnekler vardır; bu örneklerin etiketi, yani sınıfı yoktur.

2. Veri Neden Etiketsiz Olur?

Hoca bunun birkaç nedeni olduğunu anlattı.

Etiketlemeye zaman ve kaynak yetmez. Bir işletmenin müşterilerini alışveriş karakteristiklerine göre gruplamak istediğinizi düşünün: çok alanlar, az alanlar, çeşitli ürünler alanlar gibi. Önce bu kararı vermek zordur. Sonra bir milyon müşteri varsa bunları tek tek etiketlemek çok zaman alır ve bu işi yapacak kişileri bulmak da zordur. Hoca, bir market teslimat uygulamasının müşterilerini alış benzerliklerine göre gruplamayı örnek verdi: sınırsız kaynağınız olsa bile bunu elle yapmak çok zordur. Ama "harcama benzerliklerine göre grupla, benzerler aynı grupta olsun; ben sonra bu grupları gözle değerlendirip bir sonuç çıkaracağım" demek daha doğru bir çözümdür.

Örüntünün karakteristiği zamanla değişir. Banka müşterilerini harcama karakteristiklerine göre başta elle etiketlediniz diyelim. Kişi iş değiştirince ya da öğrenciyken mezun olunca bu karakteristik değişir. O zaman etiketleri sürekli güncellemek zorunda kalırsınız.

Veri gerçekten etiketsizdir. Cep telefonunuzdaki resimleri benzerliklerine göre gruplamak istediğinizi düşünün. En güzel pozu yakalamak için aynı yerde çok sayıda resim çekeriz. Bu resimler içerik olarak birbirine benzer ama birebir aynı değildir: birinde ayakta, birinde oturuyor, birinde sağa bakıyor, birinde elinizi kaldırmışsınızdır. Bunların etiketlenmesine gerek yoktur; benzerliklerine göre gruplanabilir, hatta benzerlik gruplarına göre bir hiyerarşi kurulabilir.

Sonuç olarak, vakit yokluğu, kaynak yetersizliği ya da elle etiketlemenin güç olması nedeniyle elimizde sınıf bilgisi yoksa, eğitmensiz öğrenme yöntemleriyle örnekleri birbirlerine benzerliklerine göre gruplayabiliriz. Hocanın slayttaki örnekleri:

- **Sosyal ağ analizi:** sosyal ağ kullanıcılarını kullanım benzerliklerine göre gruplama.
- **Pazar segmentasyonu (market segmentation):** müşterileri harcama benzerliklerine göre gruplayıp daha sonra bu gruplara öneride bulunma. Hoca bunun şu anda en çok yapılan uygulama olduğunu söyledi.

3. Öbeklemenin Hedefi

Eğitmensiz öğrenmede yapılan işlem, örnekleri birbirlerine benzerliklerine göre aynı gruba almaktır. Hedef şudur:

- **Sınıf içi benzerlik (within-class similarity) çok olmalı:** aynı gruptaki örnekler birbirine benzemeli.
- **Sınıflar arası benzerlik (between-class similarity) az olmalı:** farklı gruplardaki örnekler birbirine benzememeli.

"Cluster" kelimesi Türkçede "öbek", "clustering" de "öbekleme" olarak karşılanmıştır. Hoca bu karşılığı kullanabileceğimizi, ama kendisinin genelde "grup" ya da "klaster" demeyi tercih ettiğini söyledi.

Slaytta aynı örnekler iki farklı biçimde 3 gruba ayrılmıştı. Soldaki gruplamada benzerler bir arada, benzemeyenler farklı gruplardaydı; üç grup da iyi belirlenmişti. Sağdaki gruplamada da 3 grup vardı, ama bazı yerlerde birbirine çok yakın iki örnek ayrı gruplara düşmüş, birbirine pek benzemeyen iki örnek ise aynı grupta kalmıştı. Yani bir grubun içindeki mesafe, iki grup arasındaki mesafeden daha büyüktü; bu bir tersliktir. Etiketsiz verileri gruplarken olabildiğince optimum bir çözüm üretmek gerekir.

4. K-Ortalama (K-Means) Algoritması

Hoca, lisans eğitiminde veri madenciliği gibi derslerde bu yöntemle karşılaşılmış olabileceğini söyledi. K-ortalama (K-means), kolaylığı ve başarılı sonuçları nedeniyle eğitmensiz öğrenme yöntemleri içinde en çok tercih edilen yöntemdir. Yaptığı şey, etiketsiz örnekleri benzerliklerine göre K gruba yerleştirmektir. Slayttaki örnekte $K = 3$ için örnekler üç gruba ayrılmıştı.

Hoca sezgi için banka müşterilerini harcama karakteristiğine göre üç gruba ayırma örneğini kullandı: az harcayanlar, orta seviyede harcayanlar ve çok harcayanlar. Ama hangi miktarın

“az”, hangisinin “orta” ya da “yüksek” olduğunu bilmiyoruz; elimizde eşik seviyeleri yok. Sistemin bunu kendi kendine bulmasını istiyoruz.

Algoritmanın adındaki “mean” (ortalama), küme merkezinin kümedeki örneklerin ortalamasına eşit olmasından gelir. Bu yüzden küme merkezleri genelde μ ile gösterilir.

Çalışma prensibi şöyledir:

1. Başlangıçta rastgele (random) K tane küme merkezi seçilir.
2. Her örneğin her merkeze uzaklığı hesaplanır. Örnek, uzaklığın en az olduğu merkezin kümesine dahil edilir.
3. Örnekler yer değiştirdiği için her kümenin merkezi, o kümedeki örneklerin ortalaması alınarak yeniden belirlenir.
4. Merkezler değiştiyse 2. adıma dönülür; örnekler yeniden en yakın oldukları kümeye atanır ve merkezler yeniden hesaplanır.

Hoca, burada aslında bir “eğitim” olmadığını, ama iterasyonlarla gruplamanın ilerlemesini bir eğitim gibi düşünebileceğimizi söyledi.

Mesafe ölçüsü. Yakınlık için en çok Öklid (Euclidean) uzaklığı kullanılır. Çok hassasiyete gerek yoksa ve daha hızlı olsun isteniyorsa Manhattan uzaklığı kullanılır:

$$d_{\text{Öklid}}(\mathbf{x},\mathbf{y}) = \sqrt{\sum_{j=1}^{d}(x_j - y_j)^2}, \qquad d_{\text{Manhattan}}(\mathbf{x},\mathbf{y}) = \sum_{j=1}^{d}|x_j - y_j|$$

Slaytta merkezlerin iterasyonlar boyunca nasıl kaydığı adım adım gösterildi. Örneklerin hangi kümeye ait olduğu değiştikçe merkezler de kayar: bir merkez bir yöne, diğeri başka bir yöne gider. Hoca, ilk iterasyonlarda örneklerin bir düzende dururken birkaç adım sonra tamamen farklı ve oldukça mantıklı iki kümenin ortaya çıktığını gösterdi.

5. Örnek: K-Means’in Elle Çözümü

Hoca slaytta bir sayısal örnek bulamadığı için tablette elle bir örnek çözdü.

Verilen. İki özellik var: x_1 ve x_2 (örneğin kişilerin harcama miktarı ve günde ne kadar harcama yaptıkları). Yedi örnek:

Örnek	x_1	x_2
$\mathbf{x}_1$	2	1
$\mathbf{x}_2$	2	3
$\mathbf{x}_3$	3	2
$\mathbf{x}_4$	5	2
$\mathbf{x}_5$	7	2
$\mathbf{x}_6$	7	3
$\mathbf{x}_7$	8	2

$K = 2$ seçildi. Başlangıç merkezleri $\mu_1 = (3, 2)$ ve $\mu_2 = (8, 2)$ olarak alındı. Hoca bunları rastgele değil, işler biraz karışsın diye bilerek “kötü” seçtiğini söyledi. Kare ve kare kökle uğraşmamak için uzaklık olarak Manhattan kullandı; hassasiyet gerekiyorsa normalde Öklid kullanılması gerektiğini de hatırlattı.

1. iterasyon: uzaklıklar ve atama.

Örnek	$d(\cdot, \mu_1)$	$d(\cdot, \mu_2)$	Küme
$\mathbf{x}_1$	$\lvert2-3\rvert + \lvert1-2\rvert = 2$	$\lvert2-8\rvert + \lvert1-2\rvert = 7$	1
$\mathbf{x}_2$	$1 + 1 = 2$	$6 + 1 = 7$	1
$\mathbf{x}_3$	0	5	1
$\mathbf{x}_4$	2	3	1

Örnek	$d(\cdot, \mu_1)$	$d(\cdot, \mu_2)$	Küme
$\mathbf{x}_5$	4	1	2
$\mathbf{x}_6$	$4+1=5$	$1+1=2$	2
$\mathbf{x}_7$	5	0	2

Hesap sırasında hoca $\mathbf{x}_4$ için bir yerde yanlış satıra bakınca bir öğrenci uyardı; hoca düzeltti ve "hızlı yaparken hata yapma ihtimali yüksek, hata olursa uyarın" dedi. Doğru değerler tablodaki gibidir: $\mathbf{x}_4$'ün μ_1'e uzaklığı 2, μ_2'ye uzaklığı 3'tür.

Sonuç: $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4$ küme 1'e, $\mathbf{x}_5, \mathbf{x}_6, \mathbf{x}_7$ küme 2'ye gitti.

1. iterasyon: yeni merkezler.

$$\mu_1 = \left(\frac{2+2+3+5}{4},\ \frac{1+3+2+2}{4}\right) = \left(\frac{12}{4},\ \frac{8}{4}\right) = (3,\ 2)$$

$$\mu_2 = \left(\frac{7+7+8}{3},\ \frac{2+3+2}{3}\right) = \left(\frac{22}{3},\ \frac{7}{3}\right) \approx (7{,}33;\ 2{,}33)$$

μ_1 değişmedi; μ_2 ise $(8, 2)$'den $(7{,}33;\ 2{,}33)$'e kaydı.

2. iterasyon. Merkezlerden biri değiştiği için örneklerin kümesi de değişebilir; bu yüzden işlem tekrarlanır. Hoca, $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$'ün konumları nedeniyle zaten açık ara μ_1'e yakın olduğunu, asıl bakılması gerekenin $\mathbf{x}_4$ olduğunu söyledi:

$$d(\mathbf{x}_4, \mu_1) = 2, \qquad d(\mathbf{x}_4, \mu_2) = |5-7{,}33| + |2-2{,}33| \approx 2{,}33 + 0{,}33 = 2{,}67$$

$\mathbf{x}_4$ yine küme 1'de kaldı. Hiçbir örnek küme değiştirmediği için merkezler de değişmez ve algoritma durur. Sonuç: küme $1 = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4\}$, küme $2 = \{\mathbf{x}_5, \mathbf{x}_6, \mathbf{x}_7\}$.

Hoca aslında $\mathbf{x}_4$'ün ikinci adımda ikinci kümeye kaymasını göstermek istediğini, ama merkezi "çok iyi oturttuğu" için örneğin tek iterasyonda yerine oturduğunu söyledi. Örnekteki tüm uzaklık ve merkez değerleri python ile yeniden hesaplanmış ve hocanın değerleriyle tutmuştur (hoca $2{,}67$ için "2,6" dedi).

6. Durma Koşulları ve Algoritmanın Akışı

Merkezler değiştiği sürece örnekler de küme değiştirebilir. Bu yüzden iterasyonlar sürdürülür. Ne zaman durulacağı için üç seçenek vardır:

- **İterasyon sınırı:** en fazla iterasyon sayısı belirlenir (örneğin 500).
- **Merkezlerin değişmemesi:** bütün merkezler artık değişmeyene kadar devam edilir.
- **Hata eşiği:** bir önceki merkezle bir sonraki merkez arasındaki fark belirlenen ε değerinden küçük olana kadar devam edilir.

Slayttaki akışta her örneğin mesafesi hesaplanır, örnekler gruplanır, merkezler güncellenir. Kümelerde hareket varsa başa dönülür; örnekler yerlerine yerleşmişse iterasyonlar kesilir.

7. K-Means'in Sorunları: Başlangıç Merkezleri ve K'nın Seçimi

7.1 Başlangıç Merkezlerinin Etkisi

K-means her zaman başarılı bir öbekleme yapmayabilir. Bunun birden fazla nedeni vardır; biri başlangıçta merkezlerin rastgele nereden başladığıdır. Hoca tablette bir örnek çizdi: merkezleri kötü bir yerden başlattığında her örnek kendine en yakın merkeze gitti, merkezler ortalamaya göre kaydı ve sonunda "mantıklı görünen" ama aslında gözle bakıldığında beklenenden farklı bir öbekleme çıktı. Örneklerin hangi merkeze daha yakın olduğuna göre bakarsak algoritma doğru çalışmıştır; ama sonuç tamamen başlangıç değerlerine bağlı kalmıştır. Hoca, aynı veride etkin öbeklemenin farklı olduğunu gözle net görebildiğini söyledi.

Ek bilgi (derste anlatılmadı): Başlangıç merkezi sorununu azaltmak için uygulamada iki yol yaygındır. Birincisi, algoritmayı farklı rastgele başlangıçlarla birkaç kez çalıştırıp hatası en küçük sonucu seçmektir. İkincisi, merkezleri birbirinden uzak seçmeye çalışan k-means++ başlangıç yöntemidir.

7.2 K'nın Baştan Belirlenmesi

İkinci sorun K'nın yanlış seçilmesidir. Hoca bir örnekte $K = 3$ seçtiği için algoritmanın üç küme ürettiğini, ama aslında kümelerden birinin komşusuna dahil olabileceğini, yani gerçekte 2 grup bulunduğunu gösterdi. Hoca bunun bir anomali olmadığını vurguladı: uzakta duran az sayıda örneğin ayrı bir küme olması mantıklıdır, ama burada yakında bir küme varken ayrı bir küme yapılmıştır.

Böyle bir durumu anlamak için kümelerdeki örnek sayılarına bakılabilir. Örneğin 1000 örnek 3-4 kümeye yayılacaksa, bir kümede 10'dan az örnek olmamalı diye bir eşik konabilir. Bir kümedeki örnek sayısı eşiğin altındaysa ve belli bir mesafede başka bir küme varsa iki küme birleştirilir. Buradaki sıkıntı, bu eşik seviyelerinin (hangi mesafeden itibaren birleştirmenin mantıklı olduğu, bir kümede kaç örnek olması gerektiği) deneysel olarak belirlenmesidir. Hoca, işin içine böyle sabit eşikler girdiğinde çok dikkatli olmak gerektiğini, yoksa bu deneysel değerlerin ileride yanlış sonuçlara yol açabileceğini söyledi.

En önemli sıkıntı K'yı baştan belirlemek zorunda olmaktır. "Banka müşterilerini gelir grubuna göre 3 gruba ayıracağım" dediğinizde algoritma illa 3 grup üretir. Belki aslında 2 grup vardır; belki de 5 grup vardır ve siz 3 dediğiniz için bazı grupları birleştirmek zorunda kalır, küçük bir grup da açıkta kalabilir.

K'yı baştan belirlemek bazen doğrudur. Örneğin sahte haber (fake news) tespitinde haberin sahte olup olmadığını anlamak istiyorsanız iki küme vardır; "yarı sahte" diye bir şey olamaz. Ama gelir grubuna göre müşteri gruplama, başarıya göre öğrenci gruplama, benzerliklerine göre sosyal ağ kullanıcılarını gruplama gibi bugünkü gerçek iş dünyası uygulamalarının çoğunda kaç küme olduğunu bilmeyiz. O zaman en iyisi kaç grup olacağının sistem tarafından belirlenmesidir.

Soru: K'yı çok büyük, örneğin 20 seçsek, sonra bu 20 kümenin ortalamalarını birer nokta gibi alıp bunların üzerinde yeniden K-means çalıştırarak 2'ye ya da 3'e insek nasıl olur? **Cevap:** Hoca soruyu güzel bulduğunu ve K'nın otomatik olarak nasıl belirlenebileceğini, yani ön yargıyla "3 küme olsun" demek yerine sistemin etkin K'yı kendisinin nasıl bulabileceğini aradan sonra anlatacağını söyledi.

Ek bilgi (derste anlatılmadı): Öğrencinin önerdiği "önce çok sayıda küçük küme, sonra bu kümelerin merkezlerini yeniden öbekleme" fikri, büyük verilerde kullanılan iki aşamalı öbekleme yaklaşımlarına benzer. Hiyerarşik öbekleme (11-14. bölümler) de küçük kümeleri adım adım birleştirerek benzer bir sonuca gider.

8. Hata Ölçüsü: Küme İçi Kareler Toplamı

Aradan sonra hoca, K-means'i anlatırken "her örnek en yakın olduğu kümeye dahil edilir" diye basit anlattığını, ama arkada bir hata hesabı da olduğunu söyledi. C_i i'inci küme, $\mathbf{m}_i$ bu kümenin merkezi ve $\mathbf{x}$ bu kümedeki bir örnekse, bütün örnekler için kareler toplamı hatası (sum of squared error, SSE) hesaplanır:

$$SSE = \sum_{i=1}^{K} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|^2$$

Bu değer, K küme için bütün örneklerin yerleştirildikleri kümeye göre toplam hata miktarını gösterir. Amaç bu hatanın en küçük olmasıdır. Her örneği kendisine en yakın kümeye yerleştirdikçe örneğin merkeze uzaklığı küçülür; buna bağlı olarak toplam hata da küçülür. Hoca, bu formülün kendisinin uygulamalarda doğrudan çok kullanıldığını görmediğini, ama bir sonraki konu olan dirsek yönteminde aynı hatanın kullanıldığını söyledi.

9. Dirsek Yöntemi ile K'nın Belirlenmesi

Hoca problemi bir kez daha hatırlattı: elimizdeki örneklerin karakteristiği hakkında çok bilgimiz yoksa, kendi fikrimize göre "herhalde 3 küme vardır" demek, birbirine çok benzemeyen örneklerin aynı kümeye düşmesine neden olabilir.

9.1 Hocanın Çalışmasından Bir Örnek

Hoca, daha önce bahsettiği, BKM'nin gerçek verisi üzerinde yaptıkları kredi kartı sahtekarlığı çalışmasını örnek verdi. Bu çalışmada sahtekarlık işlemleri önce benzerliklerine göre gruplanmış, sonra bunlar üzerinde başka işlemler yapılmıştı. Kaç tür sahtekarlık olduğunu bilmeleri mümkün değildi. Yalnız tutara bakarak "1-10 lira arası küçük, orta büyüklükte ve çok büyük sahtekarlıklar" demek işin sadece bir boyutudur. İşin içine sahtekarlığın yapılma sıklığı, saati, zamanı, ne tür bir iş yerinde yapıldığı gibi pek çok bilgi girer. Dikkate alınması gereken özellik sayısı arttıkça bunu elle yapmak mümkün değildir. Bu yüzden farklı K değerleriyle dirsek yöntemini (elbow method) kullanarak en uygun K'yı belirlemiş ve sahtekarlıkları benzerliklerine göre gruplamışlardı.

9.2 Yöntem

1. $K = 2$'den başlanır. $K = 1$ her şeyin aynı grupta olması demektir; ondan başlamak gerekmez.
2. Bir en büyük küme sayısı belirlenir, örneğin 20. Hoca çok abartmamak gerektiğini, ama çok daha fazla küme bekleniyorsa 50, 60, 70'e kadar gidilebileceğini söyledi.
3. Her K değeri için ayrı ayrı K-means çalıştırılır ve hata hesaplanır. Hata için genellikle küme içi kareler toplamı (within-cluster sum of squares, WCSS) kullanılır.

WCSS'de C_1 kümesindeki örneklerin C_1'in merkezine uzaklıklarının kareleri toplanır; bu, birinci kümenin hatasıdır. Aynısı C_2 ve C_3 için yapılır. Üç kümenin hataları toplanınca $K = 3$ için WCSS elde edilir. Aynı hesap $K = 4$, $K = 5$ için tekrarlanır.

9.3 Hata Ne Zaman Sıfır Olur?

Hoca öğrencilere hatanın ne zaman sıfır olacağını sordu. **Soru:** Hata sıfır olur mu, olursa ne zaman? **Cevap (bir öğrenci):** Her örnek bir küme olursa hata sıfır olur. Hoca bunu onayladı: 100 örnek varsa ve $K = 100$ seçilirse, her örnek kendi kümesinin merkezi olur, merkeze uzaklığı sıfırdır, toplam hata da sıfırdır. Buradan K'yı durmadan artırmanın iyi bir şey olmadığı anlaşılır. Bir optimum nokta bulmak gerekir.

> **Hocanın vurgusu:** K'yı artırmak hatayı her zaman düşürür; amaç hatayı sıfırlamak değil, optimum K'yı bulmaktır.

9.4 Dirseği Bulmak

K arttıkça WCSS düşer. Başta düşüş büyüktür, çünkü o sırada örnekler henüz doğru kümelerde değildir. Belli bir noktadan sonra küme sayısı artsa da hata çok bariz değişmez. Düşüşün yavaşladığı bu yer, grafiğin "dirseği"dir ve optimum küme sayısını verir. Hoca, dirsekten sonra hatanın tamamen sabitlenmesini beklemediğimizi, hatanın değişmeye devam edeceğini ve sonunda sıfıra gideceğini özellikle belirtti.

Dirseği sayısal olarak bulmak için slayttaki yöntem şudur:

1. En küçük küme sayısındaki ($K = 1$ ya da başlangıç değeri) hata noktası ile en büyük küme sayısındaki (örneğin $K = 10$) hata noktası bir doğru ile birleştirilir.
2. Her K değerinin hata noktasının bu doğruya uzaklığı hesaplanır.
3. Doğruya en uzak olan nokta dirsektir; o K optimum küme sayısıdır.

Hoca, eskiden K'nın genelde baştan sabit alındığını, artık ise genellikle farklı K değerleri içinden en iyisinin bu şekilde seçildiğini söyledi.

Slaytta bunu gösteren bir sözde kod vardı. Hoca bu kodu kendi ekiplerinin, görüntüleri renk gruplarına ayırdıkları bir çalışmadan aldığını anlattı. Orada amaç bir görüntünün kaç farklı

renkten oluştuğunu bulmaktı; açık pembeyle biraz daha açık pembenin aynı kümede olması isteniyordu. Kod herhangi bir veri için geçerlidir:

```
girdi: veri, Limit (en büyük küme sayısı; kullanıcı verir)
K = 2'den Limit'e kadar:
    veri üzerinde K-means çalıştır
    bu K için WCSS hesapla
i = 1 ile i = Limit noktalarını birleştiren doğruyu çiz
her K için WCSS noktasının bu doğruya uzaklığını hesapla
uzaklığı en büyük olan K = optimum K
```

10. K-Means'in Karmaşıklığı

Hoca öğrencilere tek bir K için (dirsek yöntemi düşünülmeden, örneğin $K = 3$) K-means'in karmaşıklığını sordu: n örnek ve d özellik var. Öğrenciler önce "n çarpı d", sonra "çarpı 3" dediler. Hoca bir şeyin daha eksik olduğunu söyleyip cevabı tamamladı:

$$O(n \cdot k \cdot i \cdot d)$$

- n: örnek sayısı
- k: küme sayısı
- i: iterasyon sayısı
- d: özellik sayısı

> **Hocanın vurgusu:** Özellikle doktora yapanlar dikkat etsin; algoritma karmaşıklıkları yeterlik sınavlarında çok sorulur. Karmaşıklığı böyle yazın, sonra açıklamasını yapın.

Hocanın açıklaması şöyleydi: küme sayısı genelde azdır; özellik sayısı da sonsuza gitmez (30, 100, 500 gibi). İterasyon sayısı da azsa ve algoritma çabuk yakınsıyorsa karmaşıklık pratikte n gibi düşünülebilir. İterasyon sayısı çok fazla oluyorsa n^2'ye kadar yolu vardır. Yani pratikte karmaşıklık n ile n^2 arasında bir yerdedir.

> **Ders sonrası not:** $O(n \cdot k \cdot i \cdot d)$ ifadesinde k, i ve d sabit kabul edilirse karmaşıklık örnek sayısında doğrusaldır, yani $O(n)$'dir. "n^2'ye kadar" ifadesi ancak iterasyon sayısı i de örnek sayısıyla birlikte büyürse geçerlidir; bu, verinin yapısına bağlıdır.

11. Düz ve Hiyerarşik Öbekleme

K-means bir bölümleyici öbekleme (partitional clustering) yöntemidir; buna düz öbekleme (flat clustering) da denir. Düz öbeklemede hiyerarşi yoktur, bütün kümeler aynı seviyede kabul edilir. Örneğin bir resmi renk benzerliklerine göre mavi tonları, yeşil tonları, kırmızı tonları diye grupladığınızda bütün gruplar "renk" seviyesindedir.

İkinci tür hiyerarşik öbeklemedir (hierarchical clustering). Burada örnekler bir hiyerarşiye göre gruplanır. Slayttaki örnekte canlılar önce hayvanlar ve bitkiler diye ayrılıyor, sonra hayvanlar omurgalılar ve omurgasızlar diye, bitkiler de örneğin tohum üretenler ve spor üretenler diye alt gruplara ayrılıyordu. Slaytta yanlışlıkla kalan bir meyve resminde de elmalar ve portakallar önce kendi aralarında, sonra her biri kendi içinde gruplanıyordu. Yapıda bir hiyerarşi varsa hiyerarşik öbekleme yöntemleri kullanılabilir.

Hoca, bu dersin eğitmensiz öğrenmeye giriş olduğunu, önümüzdeki hafta düz öbeklemenin başka yöntemlerini, Gauss karışım modelleri (Gaussian mixture models) ve öz düzenleyici haritalar (self-organizing maps) gibi yöntemleri anlatacağını söyledi.

Hiyerarşik öbekleme iki şekilde yapılır:

- **Birleştirici (agglomerative):** her örnek başta ayrı bir kümedir; benzer kümeler birleştirilerek yukarı doğru gidilir. En çok kullanılanı budur.
- **Bölücü (divisive):** bütün örnekler (n örneğin tamamı) başta tek bir kümedir; bu küme alt kümelere bölünerek aşağı doğru gidilir.

Hangisinin seçileceği ihtiyaca bağlıdır.

12. Birleştirici Hiyerarşik Öbekleme ve Küme Arası Mesafeler

Slayttaki örnekte A, B, C, D, E, F örnekleri vardı. Başlangıçta her biri bir küme kabul edilir. Sonra benzerliklerine göre birleştirilir ve bu, bütün örnekler tek bir kümede olana kadar sürdürülebilir. Nerede durulacağına siz karar verirsiniz. Örneğin bir seviyede A bir kümede, B ile C bir kümede, D ile E bir kümede, F ayrı bir kümede olabilir; bu seviye yeterliyse orada dursunuz.

> **Ek bilgi (derste anlatılmadı):** Birleştirme adımlarını gösteren bu ağaç yapısına dendrogram denir. Dendrogramı belli bir yükseklikten "kesmek", o seviyedeki kümeleri verir.

Örnekler arası uzaklık için genellikle Öklid, hız isteniyorsa Manhattan kullanılır. Kümeler arası uzaklık için ise dört farklı ölçü vardır: en küçük (minimum), en büyük (maximum), ortalama (average) ve merkezler arası (mean) uzaklık.

12.1 En Küçük Uzaklık (Single Linkage)

Bu ölçü en yakın komşu (nearest neighbor) ya da tek bağlantı (single linkage) olarak da geçer. İki küme arasındaki uzaklık, iki kümedeki en yakın iki örneğin uzaklığıdır:

$$d_{\min}(C_i, C_j) = \min_{\mathbf{x}\in C_i,\ \mathbf{y}\in C_j} \|\mathbf{x}-\mathbf{y}\|$$

Hoca tablette A, B ve C kümeleriyle bir ara adım çizdi. Hangi iki kümenin birleştirileceğine karar vermek için A ile B'deki bütün örneklerin birbirine uzaklığına bakılır ve en yakın ikisi bulunur. Aynısı B ile C ve A ile C için yapılır. Bütün kombinasyonlar denendiği için bu çok yavaş bir işlemdir. Sonra bütün bu en kısa uzaklıklar içinden en kısası seçilir ve o iki küme birleştirilir.

Slaytta bu ölçünün bir zayıflığı da gösterildi: arada duran tek bir parazitli (gürültülü) örnek yüzünden aslında ayrı olması gereken iki küme birleştirilmişti; 3 küme seçildiğinde böyle bir gruplama ortaya çıkmıştı.

Hoca birleştirici öbeklemenin akışını bir kez daha özetledi: başta her küme bir örnektir (n örnek, n küme). Bütün kümeler arası uzaklıklara bakılır; gözle de görülen en kısa uzaklıktaki iki küme birleştirilir. Sonra yeniden bütün kümelerdeki bütün örneklerin birbirine uzaklığına bakılır ve en kısa uzaklıktaki iki küme birleştirilir. Böyle devam edilir.

12.2 En Büyük Uzaklık (Complete Linkage)

Bu ölçü tam bağlantı (complete linkage) ya da en uzak komşu (farthest neighbor) olarak geçer. Bu kez iki küme arasındaki uzaklık, iki kümedeki en uzak iki örneğin uzaklığıdır:

$$d_{\max}(C_i, C_j) = \max_{\mathbf{x}\in C_i,\ \mathbf{y}\in C_j} \|\mathbf{x}-\mathbf{y}\|$$

Slayttaki örnekte D_1 ile D_2 arasındaki en uzak iki örnek ve D_2 ile D_3 arasındaki en uzak iki örnek gösterilmişti. D_1 ile D_3 birbirinden çok uzak olduğu için onların en uzak mesafesi zaten daha büyüktür. Bu en uzak mesafeler hesaplandıktan sonra bunların en küçüğü seçilir, çünkü en yakın iki küme birleştirilecektir. Örnekte D_1-D_2 uzaklığı D_2-D_3 uzaklığından küçük olduğu için bir sonraki adımda D_1 ile D_2 birleştirilir.

Hoca, bu iki yöntemin son derece masraflı olduğunu söyledi: bütün kümelerdeki bütün örneklerin birbirine uzaklığını tek tek hesaplamak, sonra bunların en küçüğünü ya da en büyüğünü bulmak gerekir.

12.3 Ortalama Uzaklık (Average Linkage)

C_i kümesindeki örnekler $\mathbf{x}$, C_j kümesindeki örnekler $\mathbf{y}$ olsun. Bütün örnek çiftlerinin uzaklıkları hesaplanır, ama en küçük ya da en büyük aranmadan bu uzaklıkların ortalaması alınır:

$$d_{\text{avg}}(C_i, C_j) = \frac{1}{|C_i|\,|C_j|} \sum_{\mathbf{x} \in C_i} \sum_{\mathbf{y} \in C_j} \|\mathbf{x} - \mathbf{y}\|$$

Ortalaması en küçük olan iki küme birleştirilir.

12.4 Merkezler Arası Uzaklık (Mean / Centroid)

Hocanın "bence daha güzel" ve "en basiti" dediği bu yaklaşım K-means'e benzer. Her kümenin ortalaması alınarak merkezi (centroid) belirlenir: D_1'in, D_2'nin, D_3'ün merkezi. Sonra merkezlerin birbirine uzaklığına bakılır ve merkezleri en yakın olan iki küme birleştirilir:

$$d_{\text{mean}}(C_i, C_j) = \|\mathbf{m}_i - \mathbf{m}_j\|$$

Hoca bu ölçünün diğerlerinden farkını şöyle açıkladı: en küçük ve en büyük uzaklıkta her örneğin bütün kümelere uzaklığı hesaplanıp en küçüğü ya da en büyüğü aranır; bu masraflıdır. Ortalama uzaklıkta bütün çiftler hesaplanır ama sadece ortalaması alınır. Merkezler arası uzaklıkta ise her küme için yalnızca bir merkez vardır ve merkezlerin uzaklığı karşılaştırılır.

13. Hangi Küme Arası Mesafe Daha İyi?

Soru: Birkaç farklı yöntem var. En masraflı olanı en doğru yöntem diyebilir miyiz? Ondan farklı bir sonuç çıkarsa öbürü yanlış mı olur? **Cevap:** Hoca, deneme yapmadan bu tür durumlarda konuşmanın çok doğru olmadığını söyledi. Kendisi birleştirici öbekleme kullanacak olsa her mesafe ölçüsü için öbekleme sonuçlarında ayrı ayrı hata hesaplar, sonra hangisi iyiyse onu seçerdi. Yine de hissiyatının merkezler arası uzaklığın yeterince iyi sonuç vereceği yönünde olduğunu söyledi: merkez, örnekler hangi dağılımda olursa olsun kümenin ortasını gösterir; merkezlerin yakınlığı da kümelerin yakınlığını başarıyla anlatır. Ufak tefek kaçaklar olabilir, ama bu yaklaşımın hem daha hızlı hem daha etkin olacağını tahmin ettiğini söyledi.

> **Ders sonrası not:** Hoca burada "çok ezbere bir şey söyleyip de mean distance'a göre hesaplayın demek doğru bir yaklaşımdır" dedi. Cümlenin devamından ("çünkü … elde edeceğiniz sonuç değişebilir") kastının tersi olduğu, yani ezbere tek bir ölçü önermenin doğru olmadığı anlaşılıyor.

Hoca, makine öğrenmesinde sonucun elinizdeki örneklerin dağılımına, özellik sayısına ve özelliklerin durumuna göre değişebileceğini, bu yüzden temkinli konuşmak gerektiğini vurguladı. Daha önce anlattığı performans metriklerine (accuracy, precision, recall vb.) bakarak karar ağacı mı yoksa k-en yakın komşu mu kullanılacağına karar verildiği gibi, parametre seçimi de aynı yolla yapılır. Birleştirici öbeklemede hangi mesafe ölçüsünün kullanılacağı da farklı ölçülerle hesaplayıp en iyi sonucu veren seçilerek belirlenebilir. Hocanın kendi "çok hassas değilse Manhattan kullanın" önerisi bile yuvarlak bir sözdür, çünkü hassasiyetin yeterli olup olmadığına da deneyerek karar verilir.

> **Hocanın vurgusu:** Bu işlerde kesin bir doğru yok; sistemler örneklere çok bağlıdır. Hangi yöntemin ya da parametrenin iyi olduğu denenerek bulunur.

Soru (bir öğrenci): Etiket olmadığı için ortalamayı ne kadar doğru bulduğumuz da ayrı bir problem değil mi? **Cevap:** Hoca bunu gerçek dünyada çok yaşadıklarını söyledi: K'yı 5 alınca bir sonuç, 6 alınca başka bir sonuç çıkıyor ve neyin en etkin olduğunu ölçmek gerçekten zor. Etiket yoksa hata hesaplansa bile gerçek sonucun ne olduğu bilinmez; ama mantıken iyi sonuç çıkacağını düşündüğünü söyledi.

14. Bölücü Hiyerarşik Öbekleme

Bölücü (divisive) hiyerarşik öbeklemede tek bir kümeyle, yani bütün örneklerle başlanır. Sonra örnekler benzerliklerine göre bölünür. K-means mantığıyla düşünülürse önce iki kümeye bölünür: iki merkez alınır, benzer örnekler bu iki kümeye yerleştirilir. Sonra bu kümelerin her biri yine ikiye bölünür. Böyle her küme tek örnek kalana kadar gidilebilir.

Hangi seviyede kesileceğine yine siz karar verirsiniz. Örneğin "hatanın en uygun olduğu yeri burası olarak buldum, daha fazla bölünmesini istemiyorum" diyebilirsiniz. Hoca, hiyerarşik öbeklemelerde genellikle hiyerarşiyi önceden bildiğinizi söyledi. Canlılar örneğinde omurgalılar ve omurgasızlar seviyesinde kesmek istiyorsanız ikinci seviyede kesersiniz. Daha ayrıntı istiyorsanız, örneğin köpekleri de türlerine göre ayırmak istiyorsanız daha aşağı inersiniz. Hoca bu yüzden sistemin "çok da başı boş bir sistem olmadığını" söyledi.

Hoca görüntü işlemeden bir örnek verdi: görüntüleri bölütlemede (segmentation) pikseller renk özelliklerine göre bölünür. Bir seviyede bir kümedeki bütün pikseller aynı renkteyse orada kesilir. Hâlâ açık maviler ile koyu maviler bir aradaysa, yani küme kendi içinde homojen değilse bir kademe daha bölünür. Aynı mantıkla bölücü hiyerarşik öbeklemede de karar, sınıf içi benzerliğin çok ve sınıflar arası benzerliğin az olmasına göre verilir: kareler toplamı hatası hesaplanır ve hata yeterince küçükse daha alt seviyelere inilmez.

15. Uygulama Alanları

Hoca, uygulamada hiyerarşik öbeklemeden çok düz öbeklemenin, yani K-means türü, eşit seviyedeki örneklerin öbeklenmesinin kullanıldığını söyledi. Ardından uygulama örnekleri verdi:

- **Görüntüler:** İlk yıllarda K-means ve benzeri eğitmensiz yöntemler görüntülerde, renk benzerliklerine göre gruplama için çok kullanılıyordu. Örneğin doğa resimlerinde mavi (deniz, gökyüzü) ve yeşil ağırlıktadır; şehir resimlerinde gri ve araçlar ağırlıktadır. Neyle karşılaşılacağı bilinmediği için önceden öğretmek mümkün değildir. Resimler renk benzerliklerine göre gruplanır, elde doğa resimleri ve şehir resimleri diye iki grup oluşur. Etiketleme gruplamadan sonra yapılır: gruptaki örneklere bakılıp "bunlar şu duruma ait örnekler" denir. Pek çok çalışmada böyle yapılır.
- **Sahte haber tespiti:** Twitter gibi ortamlarda yayılıp ortalığı hareketlendiren, sonra gerçek olmadığı anlaşılan haberler son yıllarda önem kazandı. Haberler başlığına ve içeriğine göre benzerliklerine göre gruplanarak sahte haberin anlaşılması için kullanılıyor.
- **Spam e-posta tahmini:** E-postalar benzerliklerine göre gruplanıyor; bu gruplama sonuçları bir sonraki aşamada eğitmenli öğrenme sistemine bilgi olarak veriliyor.
- **Doküman gruplama:** Eskiden dokümanlar tek tek spor, magazin, siyaset diye etiketlenirdi. Artık K bile bilinmeden dokümanlar benzerliklerine göre gruplanıyor; gerekiyorsa gruplar sonra etiketleniyor.
- **Pazarlamada kişiselleştirme:** Hocanın "en çok parladığı konu" dediği alan budur. Eskiden veri madenciliğinde "üniversite bitirmiş, büyükşehirde yaşayan kişiler şöyle harcama yapar" gibi zorlama gruplar kurulurdu. Sonra bunun pek doğru olmadığı, farklı özelliklerdeki insanların aynı gruplarda yer alabildiği görüldü. Şimdi örneğin bir teslimat uygulamasının müşterileri önce farklı K değerleriyle bakılarak gruplanıyor, sonra her gruba farklı ürünler öneriliyor. Ya da harcama profiline göre müşteriyi bir basamak yukarı taşıyacak ürünler önerilmeye çalışılıyor.
- **Müşteri kaybı analizi (churn analysis):** Müşteriler gruplanarak "bu müşteriyi kaybetmek üzere miyim" sorusuna cevap aranıyor; sadık ve sadık olmayan müşteriler benzerliklerine göre gruplanıyor.

Hoca, eğitmensiz öğrenme yöntemlerinin şu anda aktif olarak çok kullanıldığını söyledi.

16. Önce Öbekle, Sonra Sınıflandır: Veri Analizinin Önemi

Hoca soru sorulmamasından yakındı; sınıfta yüz yüze derslerde daha çok soru sorulduğunu, akla takılanı sormanın konuyu pekiştirdiğini söyledi. Bunun üzerine bir öğrenci sahte haber tespitiyle ilgili bir soru sordu.

Soru: Örneğin yakında seçimler var ve seçimlerle ilgili çok fazla sahte haber çıkıyor. 1000 haberden 900'ü sahteyse, bu konuda çıkan herhangi bir haber de sahte olarak değerlendirilmez mi? Özellikle öbekleme kullandığımızda, veri etiketsizken, sistem sahte haberin yoğun olduğu konuya doğru eğilmez mi? **Cevap:** Hoca önce sadece başlığa bakarak, yani az bilgiyle yapılan sınıflandırmanın ne kadar doğru olduğunu sorgulamak gerektiğini söyledi. Başlıkta az kelime vardır; az bilgiden çıkarılan sonucun başarısı çok yüksek olmaz. Twitter gibi kelime sayısı kısıtlı metinlerde bile sınıflandırma başarısı sıkıntılıdır, başlıkta durum daha da zordur. Yanlış yapması zaten beklenir.

Öğrencinin asıl sorusuna ise ezbere cevap verilemeyeceğini söyledi: bunun olup olmadığını görmek için yapıp bakmak gerekir. Hoca, makine öğrenmesinin en eğlenceli kısmının bu olduğunu, işin tek adımda bitmediğini anlattı. Uygulamada öbekleme serbest bırakılmaz: veri öbeklenir, sonra bu kümeler bir adım sonra sınıflandırıcıda değerlendirilir. İyi öbekleme yapılamadığı görülürse nedenine bakılır: belki problem öbeklemeye hiç uygun değildir. Hoca, önce öbekleyip sonra sınıflandırmanın şu anda çok kullanıldığını söyledi.

Hoca, makine öğrenmesinde uzaktan bakarak bir şeyin doğrusunu söylemenin mümkün olmadığını bir benzetmeyle anlattı: temel problemlerde bütün doktorlar aynı teşhisi koyar, ama belirtileri farklı ya da az görülen bir hastalıkta bir doktor daha iyi teşhis koyar, çünkü sebep-sonuç ilişkisini iyi değerlendirir. Makine öğrenmesi de böyledir. Hoca, özellikle işletmelerde, değerlendirdiği TEYDEB projelerinde ve Ar-Ge merkezi değerlendirmelerinde, veriyi Python'da bir sınıflandırıcıya ya da derin öğrenmeye verip çıkan sonucu yeterli gören yaklaşımı çok gördüğünü söyledi. Başarılı çalışmalarda ise analiz önemlidir: başarısız olduysa neden olduğuna, hangi örnek türlerinde ve hangi özelliklerde sorun çıktığına bakılır.

Örnek: Yurt dışı arama sahtekarlığı. Hoca, değerlendirdiği bir TÜBİTAK projesinden yöntem ayrıntısına girmeden basit bir kısmı anlattı. Proje, yurt dışından gelen aramaların sahte olup olmadığını anlamaya yönelikti. Bir servis sağlayıcı yurt dışından gelen bir arama için operatöre örneğin 1 euro, yurt içi arama için 10 cent öder. Bazı servis sağlayıcılar Almanya'dan gelen aramayı yurt içi aramaymış gibi gösterip 10 cent öder. Çok sayıda servis sağlayıcı olduğu için hangisinin gerçek, hangisinin sahte olduğunu ayırmak kolay değildir.

Proje ekibi, yakaladıkları ve yurt dışı arama olduğu belli olan aramaları "sahte" diye etiketlemiş, geri kalanın tamamına "sahte değil" demişti. Hoca öğrencilere bunun doğru olup olmadığını sordu. **Cevap (bir öğrenci):** Yanlış; geri kalanların içinde de sahte olanlar olabilir. Hoca bunu onayladı: geri kalanların içinde tespit edilememiş sahte aramalar olabilir. "Sahte değil" sınıfı için gerçekten sahte olmadığı bilinen temiz örnekler seçmek gerekir. Hoca bunu toplantıda söylediğinde operatör tarafının "biz buna hiç böyle bakmamıştık" diye çok şaşırdığını anlattı. Böyle bir etiketlemede başarının düşük çıkması da doğaldır.

> **Hocanın vurgusu:** Veriyi çok iyi analiz edin. Bir veriyi alıp karar ağacıyla sınıflandırıp bir sonuç elde etmek çok yapılan bir şeydir. Önemli olan, özellikle yüksek lisans ve doktora tezlerinde, veriyi nasıl analiz ettiğiniz ve nasıl değerlendirdiğinizdir.

Dersin sonunda öğrenciler ödevle ilgili sorular sordu. Akademik açıdan önemli kısmı şudur: entropi gibi hesaplar için hazır kütüphane fonksiyonlarının kullanılabileceğini, entropinin "olasılık çarpı logaritma olasılık" toplamı olduğunu ve bunun elle de yazılabileceğini söyledi.

Dersin Toparlaması

Bu derste eğitmensiz öğrenmeye giriş yapıldı. Etiketli veri olmadığında, örneklerin sınıf içi benzerliği yüksek ve sınıflar arası benzerliği düşük olacak biçimde gruplanması amaçlanır. En çok kullanılan yöntem olan K-means, rastgele seçilen K merkezden başlayıp örnekleri en yakın merkeze atar ve merkezleri küme ortalamasıyla günceller; bu işlem merkezler değişmeyene kadar sürer. Elle çözülen örnekte $K = 2$ ve Manhattan uzaklığıyla algoritma tek iterasyonda yerine oturdu. K-means'in iki temel sorunu başlangıç merkezlerine bağımlılık ve K'nın baştan seçilmesidir. K'yı otomatik seçmek için farklı K değerlerinde küme içi kareler toplamı hesaplanır ve dirsek yöntemiyle optimum nokta bulunur. Algoritmanın karmaşıklığı $O(n \cdot k \cdot i \cdot d)$'dir.

Ardından hiyerarşik öbekleme anlatıldı. Birleştirici yöntem tek örnekli kümelerden başlayıp en yakın kümeleri birleştirir; bölücü yöntem tek kümeden başlayıp böler. Küme arası uzaklık için en küçük, en büyük, ortalama ve merkezler arası uzaklık ölçüleri vardır; hoca en pratik olarak merkezler arası uzaklığı öne çıkardı ama doğru ölçünün deneyerek seçilmesi gerektiğini vurguladı. Ders, öbeklemenin uygulama alanları ve veriyi iyi analiz etmenin önemiyle kapandı. Önümüzdeki hafta düz öbeklemenin başka yöntemleri (Gauss karışım modelleri, öz düzenleyici haritalar) anlatılacak.

Ders Sonrası Notlar

Ders sonrası not: Hoca hata ölçüsü için “within cluster sum of error” dedi ve bir yerde “sınıflar arası toplam hatayı hesaplıyoruz” ifadesini kullandı. Hesaplanan şey küme içi kareler toplamıdır (within-cluster sum of squares, WCSS); her örneğin kendi kümesinin merkezine uzaklığının karesi toplanır. Altyazıda “BCSS” ve “VCSS” olarak geçen kısaltmanın doğrusu WCSS’dir.

Ders sonrası not: Altyazıda bozuk geçen terimlerin doğru yazımları: “K-Minis/K-MİS/Kamiiz” = K-means, “öğret/öplik” = Öklid (Euclidean), “Manhattan Business” = Manhattan distance, “elbol/evrol” = elbow, “eglomeratif/agnomeratif” = agglomerative, “devizli” = divisive, “complete linkage by the snaver” = complete linkage (farthest neighbor), “Gözler, mixture” = Gaussian mixture, “çördü analizi” = churn analizi, “TEDEB” = TEYDEB, “Yerinde AGE Merkezi” = Ar-Ge merkezi, “multivir perceptron” = multilayer perceptron, “yapay neuronaları” = yapay sinir ağları.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.